

10/04/26

Ex. No: 10a

First Fit Contiguous Memory Allocation

Aim:

To implement the First Fit contiguous memory allocation technique in C and calculate internal fragmentation.

Sample Input:

5

100 500 200 300 600

4

212 417 112 426

Sample Output:

Enter number of memory blocks:

Enter size of each block:

Enter number of processes:

Enter size of each process:

Process 1 of size 212 is allocated to Block 2 of size 500 with

Fragment 288.

Process 2 of size 417 is allocated to Block 5 of size 600 with

Fragment 183.

Process 3 of size 112 is allocated to Block 3 of size 200 with

Fragment 88.

✓ Process 4 of size 426 is not allocated.

Result:

The program successfully allocates processes using the First Fit method and calculates internal fragmentation, while indicating

o 10/04/26

Ex.No: 10 b

Best Fit Contiguous Memory Allocation

Aim:

To implement the Best Fit contiguous memory allocation technique in C and calculate internal fragmentation.

Sample Input:

3

100 200 300

4

90 150 250 400

Sample Output:

Enter number of memory blocks :

Enter size of each block :

Enter number of processes :

Enter size of each process :

Process 1 of size 90 is allocated to Block 1 of size 100 with
Fragment 10

Process 2 of size 150 is allocated to Block 2 of size 200 with
Fragment 50

~~Process 3 of size 250 is allocated to Block 3 of size 300 with
Fragment 50~~

Process 4 of size 400 is not allocated.

Result:

The program successfully allocates processes using the Best Fit method, minimizing internal fragmentation and identifying unallocated processes.

FACE Prep

Operating System - Coding | RED | 3 Feb

Back To Practice

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the size of various memory blocks and the size of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

- The first line contains an integer representing the number of memory blocks.
- The second line contains an integer representing the size of each memory block.
- The third line contains an integer representing the number of processes.
- The fourth line contains an integer representing the size of each process.

Output Format

- For each process, the program prints whether the process is allocated or not.
- If allocated, it displays the process number, process size, block number, block size, and the internal fragmentation (block size minus process size).
- If not allocated, it indicates that the process cannot be allocated.

Sample Input 1

```
5
5
5
500 500 500 500 500
4
100 400 300 400
```

Sample Output 1

Enter number of memory blocks:

Enter size of each block:

Enter number of processes:

Enter size of each process:

Process 0 of size 100 is allocated to Block 0 of size 500 with Fragment 400

Process 1 of size 400 is allocated to Block 0 of size 500 with Fragment 100

Process 2 of size 300 is allocated to Block 3 of size 500 with Fragment 200

Process 3 of size 400 is not allocated

Select Language: C (GCC 9.4.0)

```
1 // Include headers
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main() {
6     int n;
7     printf("Enter number of memory blocks (n): ");
8     scanf("%d", &n);
9     int blocks[n];
10    int originalblocks[n];
11    printf("Enter size of each block (b): ");
12    for(int i = 0; i < n; i++) {
13        scanf("%d", &blocks[i]);
14        originalblocks[i] = blocks[i];
15    }
16    printf("Enter number of processes (p): ");
17    scanf("%d", &p);
18    int processes[p];
19    printf("Enter size of each process (p): ");
20    for(int i = 0; i < p; i++) {
21        scanf("%d", &processes[i]);
22    }
23    for(int i = 0; i < p; i++) {
24        int allocated = 0;
25        for(int j = 0; j < n; j++) {
26            if(blocks[j] >= processes[i]) {
27                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d",
28                    i + 1, processes[i], j, blocks[j],
29                    blocks[j] - processes[i]);
30                originalblocks[j] -= processes[i];
31                allocated = 1;
32                break;
33            }
34        }
35        if(!allocated) {
36            printf("Process %d of size %d is not allocated",
37                i + 1, processes[i]);
38        }
39    }
40    return 0;
41 }
```

Test Cases

Run Sample Cases

Custom Test Case

Sample Test Cases

Test Case - 1

Test Case - 2

FACE Prep

Operating System - Coding | RED | 3 Feb

Back To Practice

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

- The first line contains an integer representing the number of memory blocks.
- The second line contains the size of the memory blocks.
- The third line contains an integer representing the number of processes.
- The fourth line contains the size of the processes.

Output Format

- For each process, display whether it is allocated or not.
- If allocated, display the block number, block size, and internal fragmentation.
- If not allocated, display that the process could not be allocated.

Example 1

Sample Input 1

```
5
5
5
500 500 500 500 500
4
100 400 300 400
```

Sample Output 1

Enter number of memory blocks:

Enter size of each block:

Enter number of processes:

Enter size of each process:

Process 0 of size 100 is allocated to Block 4 of size 500 with Fragment 400

Process 1 of size 400 is allocated to Block 2 of size 500 with Fragment 100

Process 2 of size 300 is allocated to Block 3 of size 500 with Fragment 200

Process 3 of size 400 is allocated to Block 0 of size 500 with Fragment 100

Select Language: C (GCC 9.4.0)

```
1 // Include headers
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main() {
6     int n;
7     printf("Enter number of memory blocks (n): ");
8     scanf("%d", &n);
9     int blocks[n];
10    int originalblocks[n];
11    printf("Enter size of each block (b): ");
12    for(int i = 0; i < n; i++) {
13        scanf("%d", &blocks[i]);
14        originalblocks[i] = blocks[i];
15    }
16    printf("Enter number of processes (p): ");
17    scanf("%d", &p);
18    int processes[p];
19    printf("Enter size of each process (p): ");
20    for(int i = 0; i < p; i++) {
21        scanf("%d", &processes[i]);
22    }
23    for(int i = 0; i < p; i++) {
24        int bestIndex = -1;
25        for(int j = 0; j < n; j++) {
26            if(blocks[j] >= processes[i]) {
27                if(bestIndex == -1 || blocks[j] < blocks[bestIndex]) {
28                    bestIndex = j;
29                }
30            }
31        }
32        if(bestIndex != -1) {
33            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d",
34                i + 1, processes[i], bestIndex, blocks[bestIndex],
35                blocks[bestIndex] - processes[i]);
36            originalblocks[bestIndex] -= processes[i];
37            else {
38                printf("Process %d of size %d is not allocated",
39                    i + 1, processes[i]);
40            }
41        }
42    }
43    return 0;
44 }
```

Test Cases

Run Sample Cases

Custom Test Case

Sample Test Cases

Test Case - 1

Test Case - 2

FACE Prep

Operating System - Coding | RED | 3 Feb

Back To Practice

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

- The first line contains an integer representing the number of memory blocks.
- The second line contains the size of the memory blocks.
- The third line contains an integer representing the number of processes.
- The fourth line contains the size of the processes.

Output Format

- For each process, display whether it is allocated or not.
- If allocated, display the block number, block size, and internal fragmentation.
- If not allocated, display that the process could not be allocated.

Example 1

Sample Input 1

```
5
5
5
500 500 500 500 500
4
100 400 300 400
```

Sample Output 1

Enter number of memory blocks:

Enter size of each block:

Enter number of processes:

Enter size of each process:

Process 0 of size 100 is allocated to Block 4 of size 500 with Fragment 400

Process 1 of size 400 is allocated to Block 2 of size 500 with Fragment 100

Process 2 of size 300 is allocated to Block 3 of size 500 with Fragment 200

Process 3 of size 400 is allocated to Block 0 of size 500 with Fragment 100

Select Language: C (GCC 9.4.0)

```
1 // Include headers
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main() {
6     int n;
7     printf("Enter number of memory blocks (n): ");
8     scanf("%d", &n);
9     int blocks[n];
10    int originalblocks[n];
11    printf("Enter size of each block (b): ");
12    for(int i = 0; i < n; i++) {
13        scanf("%d", &blocks[i]);
14        originalblocks[i] = blocks[i];
15    }
16    printf("Enter number of processes (p): ");
17    scanf("%d", &p);
18    int processes[p];
19    printf("Enter size of each process (p): ");
20    for(int i = 0; i < p; i++) {
21        scanf("%d", &processes[i]);
22    }
23    for(int i = 0; i < p; i++) {
24        int bestIndex = -1;
25        for(int j = 0; j < n; j++) {
26            if(blocks[j] >= processes[i]) {
27                if(bestIndex == -1 || blocks[j] < blocks[bestIndex]) {
28                    bestIndex = j;
29                }
30            }
31        }
32        if(bestIndex != -1) {
33            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d",
34                i + 1, processes[i], bestIndex, blocks[bestIndex],
35                blocks[bestIndex] - processes[i]);
36            originalblocks[bestIndex] -= processes[i];
37            else {
38                printf("Process %d of size %d is not allocated",
39                    i + 1, processes[i]);
40            }
41        }
42    }
43    return 0;
44 }
```

Test Cases

Run Sample Cases

Custom Test Case

Sample Test Cases

Test Case - 1

Test Case - 2